The JSON format represents data in simple name-value pairs. For our example, the book's title (name) is *Effective Java,* and its author (as a value) is Joshua Bloch. We can add additional fields (including nested objects) to the document: for example, we've added `prices` as a nested object.

Now that we have an idea of our bookstore and its data model, it is time to start populating Elasticsearch with a set of books to create an inventory. We do this in the next section.

2.1.2 Indexing documents

To work with the server, we need to get the client's data indexed into Elasticsearch. There are a few ways we can bring the data into Elasticsearch in the real world: creating an adapter to pipe the data from a relational database, extracting the data from a file system, streaming events from a real-time source, and so on. Whichever choice we make for our data source, we invoke Elasticsearch's RESTful APIs from the client application to load the data into Elasticsearch.

Any REST-based client (cURL, Postman, advanced REST client, HTTP module for JavaScript/NodeJS, programming language SDKs, etc.) can help us talk to Elasticsearch via the API. Fortunately, Elastic has a product that does exactly this (and more): Kibana. Kibana is a web application with a rich user interface, allowing users to index, query, visualize, and work with data. *This is our preferred option, and we use Kibana extensively in this book.*

> **RESTful access**
>
> Communication with Elasticsearch takes place via JSON-based RESTful APIs. In the current digital world, you are highly unlikely to find a programming language that doesn't support accessing RESTful services. In fact, designing Elasticsearch with APIs exposed as JSON-based RESTful endpoints was a smart choice, because it enables programming-language-agnostic adoption.

Document APIs

Elasticsearch's document APIs help with creating, deleting, updating, and retrieving documents. The APIs are accessible via HTTP transport using RESTful actions. That is, to index a document, we need to use an HTTP `PUT` or `POST` (more on `POST` later) on an endpoint. Figure 2.2 shows the syntax of the full URL format for an HTTP `PUT` method.

As you can see, the URL is composed of several elements:

- An HTTP action such as `PUT`, `GET`, or `POST`
- Server's hostname and port
- Index name
- Document API's endpoint (`_doc`)
- Document ID
- Request body